Projeto - Cidades ESGInteligentes

EcoImpact

Documentação técnica DevOps

CI/CD, containerização e orquestração de uma API ESG sobre MongoDB

Integrantes

[nome completo - RM]
[nome completo - RM]

Fase: Navegando pelo mundo DevOps

1. Visão geral da aplicação

O **EcoImpact** monitora a emissão de carbono de usuários e colaboradores: registra atividades com fator de emissão de CO2e, compara o total mensal com a meta de redução, gera alertas quando a meta é ultrapassada e abate as compensações (plantio, créditos, energia solar). O modelo original em Oracle/PL-SQL (5 tabelas e 4 triggers) foi migrado para MongoDB (5 collections). Neste desafio, as regras das triggers foram levadas para a camada de aplicação em uma API REST Node.js/Express, e o conjunto recebeu o ciclo DevOps completo.

Collection (MongoDB)	Equivale a (Oracle)	Papel
usuarios_esg	(nova)	Usuários e colaboradores
categorias_emissao	CATEGORIA_EMISSAO	Atividades, fator de emissão e escopo GHG
registros_emissao	REGISTRO_EMISSAO	Lançamentos com kg de CO2e e nível de impacto
metas_sustentabilidade	META_SUSTENTABILIDADE + ALERTA_EMISSAO	Meta mensal, totais, saldo e alertas embutidos
compensacoes_carbono	COMPENSACAO_CARBONO	Ações de compensação de carbono

Trigger original	Agora na aplicação
TRG_CLASSIFICA_NIVEL_IMPACTO	classificarNivelImpacto(): BAIXO (< 10 kg), MEDIO (10 a 50 kg), ALTO (> 50 kg)
TRG_ATUALIZA_META_EMISSAO	recalcularMeta(): total emitido e saldo do mês
TRG_ATUALIZA_COMPENSACAO	recalcularMeta(): total compensado e saldo
TRG_ALERTA_EMISSAO	gerarAlertaSeExcedeu(): alerta LIMITE_EXCEDIDO embutido na meta

Rota	Descrição
GET /health	Saúde da aplicação e do banco (ping no MongoDB)
GET /api/usuarios, /api/categorias, /api/metas, /api/compensacoes	Consultas (com filtros por escopo, usuário e mês)
GET / POST /api/registros	Lista / lança emissão (calcula kg, impacto, meta e alerta)
POST /api/compensacoes	Lança compensação (atualiza total compensado e saldo)
GET /api/resumo	Total emitido por usuário, do maior para o menor

Testes: 15 testes (node:test) cobrem as regras de negócio e a API (carga inicial, health, filtros, emissões, compensações, alertas e validações), sobre MongoDB em memória (mongo-mock). O pipeline também executa o script original do EcoImpact.

2. Pipeline CI/CD

Ferramenta: GitHub Actions (*.github/workflows/ci-cd.yml*), com GitHub Container Registry (ghcr.io) para as imagens e GitHub Environments (*staging* e *production*) para os deploys.

Etapa	Gatilho	Lógica
build-test	push e pull request	npm ci (build), npm test (15 testes) e execução do script original do EcoImpact; log publicado como artefato.
docker-image	push em main/develop, após build-test	Build da imagem com Buildx (cache) e push ao ghcr.io com tags latest e SHA do commit.
deploy-staging	após docker-image	scripts/deploy.sh staging: sobe app + MongoDB via Compose (porta 8081) e executa o smoke test.
deploy-production	somente main, após staging	scripts/deploy.sh production (porta 8082) e smoke test. Aprovação manual configurável no environment.

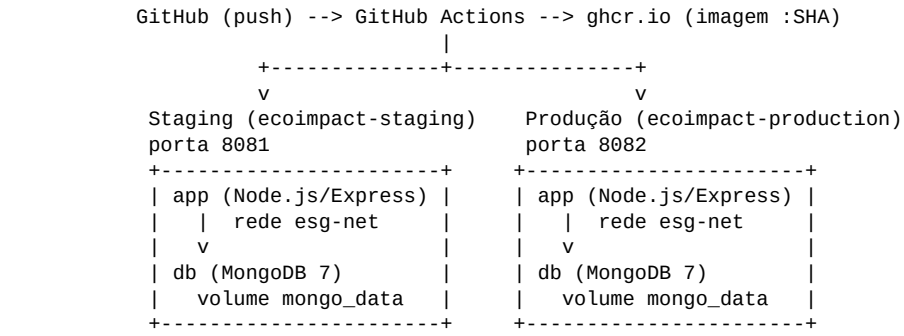
Lógica de promoção. A mesma imagem, identificada pelo SHA do commit, é promovida de staging para produção; só as variáveis de ambiente mudam (env/staging.env e env/production.env). Se qualquer etapa falhar, as seguintes não executam: um teste quebrado impede a imagem, e um smoke test quebrado em staging impede produção.

Smoke test. O script scripts/smoke-test.sh aguarda o /health responder UP (até 40 tentativas), o que inclui o ping no MongoDB real, e valida /api/categorias, /api/metas e /api/resumo.

Segredos. As senhas do banco vêm dos secrets STAGING_DB_PASSWORD e PRODUCTION_DB_PASSWORD; nada sensível é versionado (o .env é ignorado pelo Git e o repositório traz apenas o .env.example).

3. Docker

Arquitetura (um conjunto por ambiente, isolado por nome de projeto, rede, volume e porta):



Dockerfile (multi-stage). Estágio 1 (node:20-alpine) instala só as dependências de produção com npm ci, usando o cache de camadas dos manifests. Estágio 2 (node:20-alpine) leva as dependências e o código da API, roda com o usuário não-root *node* e define HEALTHCHECK pelo /health. Banco, credenciais e ambiente vêm por variáveis de ambiente.

docker-compose.yml. Serviços *app* e *db* (MongoDB 7 com autenticação); rede bridge *esg-net* (o banco não publica porta no host); volume nomeado *mongo_data* para persistência; variáveis DB_*, APP_ENV, APP_PORT e SEED_ON_START; *depends_on* com *condition: service_healthy* para a aplicação só iniciar com o banco pronto.

Comandos usados

```
cp .env.example .env                    # define DB_PASSWORD
docker compose up -d --build            # constrói a imagem e sobe app + banco
docker compose ps                       # estado e healthchecks
docker compose logs -f app              # logs da aplicação
curl http://localhost:3000/health
docker compose down [-v]                # encerra (-v remove o volume)

# Deploy por ambiente (usado pelo pipeline)
IMAGE=ghcr.io/<org>/<repo>:<sha> DB_PASSWORD=*** scripts/deploy.sh staging
IMAGE=ghcr.io/<org>/<repo>:<sha> DB_PASSWORD=*** scripts/deploy.sh production
```

Imagem criada: ghcr.io/<usuario>/<repositorio>, tags *latest* e o SHA do commit.

4. Prints do pipeline rodando

Link da execução no GitHub Actions: [colar a URL aqui]

Inserir print aqui

Visão geral do pipeline concluído (4 jobs verdes)

(arquivo sugerido: docs/prints/01-pipeline-geral.png)

Inserir print aqui

Job build-test: build e testes passando

(arquivo sugerido: docs/prints/02-build-testes.png)

Inserir print aqui

Imagem publicada no GitHub Container Registry

(arquivo sugerido: docs/prints/03-imagem-ghcr.png)

Inserir print aqui

Job deploy-staging concluído

(arquivo sugerido: docs/prints/04-deploy-staging.png)

Inserir print aqui

Job deploy-production concluído

(arquivo sugerido: docs/prints/05-deploy-producao.png)

Saída do npm test (execução local, arquivo docs/logs/npm-test-local.log):

```
ok 1 - carga inicial é idempotente e cria as 5 collections populadas
ok 2 - GET /health responde UP quando o banco está acessível
ok 3 - GET /health responde 503 quando o banco está fora
ok 4 - GET /api/categorias lista e filtra por escopo GHG
ok 5 - metas da carga inicial têm totais, saldo e alerta calculados (antes: triggers)
ok 6 - POST /api/registros calcula kg, classifica o impacto e atualiza a meta e o alerta
ok 7 - POST /api/compensacoes atualiza total compensado e saldo da meta
ok 8 - GET /api/resumo ordena usuários pelo total emitido (maior primeiro)
ok 9 - validações devolvem 400 / 404
ok 10 - categoria inativa não aceita novos registros
ok 11 - classificarNivelImpacto respeita as faixas BAIXO / MEDIO / ALTO
ok 12 - calcularKgCo2e multiplica quantidade pelo fator e arredonda em 4 casas
ok 13 - anoMes devolve o período no formato AAAA-MM
ok 14 - gerarAlertaSeExcedeu só gera alerta quando o total passa da meta
ok 15 - calcularSaldo = emitido - compensado
# tests 15
# pass 15
# fail 0
```

5. Prints dos ambientes staging e produção

Inserir print aqui

Staging funcionando (porta 8081): /health e /api/metas

(arquivo sugerido: docs/prints/06-staging-funcionando.png)

Inserir print aqui

Produção funcionando (porta 8082): /health e /api/metas

(arquivo sugerido: docs/prints/07-producao-funcionando.png)

6. Desafios encontrados e como foram resolvidos

Desafio	Solução
O projeto original era um script isolado (sobre mongo-mock), sem aplicação nem servidor para conteinerizar.	Criada uma API Express sobre o mesmo modelo de dados, preservando o script original inalterado em src/ e reaproveitando os mesmos dados na carga inicial.
As regras de negócio estavam em triggers PL/SQL, que não existem no MongoDB.	Regras movidas para funções puras (regras.js) e para a camada de serviço (servico.js), com testes unitários.
Os testes não podiam depender de um MongoDB externo no CI.	Banco em memória com mongo-mock nos testes; o MongoDB real é exercitado no deploy de staging, pelo smoke test.
O runner de testes não encerrava: o mongo-mock mantém um intervalo ativo enquanto o banco está aberto.	Os testes fecham o banco em memória (db.close()) ao terminar, e o processo encerra normalmente.
A aplicação subia antes do MongoDB ficar pronto.	Healthcheck no banco (ping) e depends_on com condition: service_healthy no Compose.
Manter staging e produção no mesmo host sem conflito de portas, volumes ou redes.	Um projeto Compose por ambiente (COMPOSE_PROJECT_NAME), com porta própria (8081 e 8082) e arquivos env/staging.env e env/production.env.
Não versionar senhas e ainda assim automatizar o deploy.	Senhas via GitHub Secrets; no repositório só o .env.example. O Compose exige DB_PASSWORD, e as credenciais vão separadas da URI de conexão.
Garantir que uma versão com defeito não chegue à produção.	Promoção em cadeia: testes, imagem, staging + smoke test, produção. Aprovação manual opcional via Required reviewers.

Nota: complemente esta tabela com os problemas que a equipe enfrentar ao executar o pipeline no GitHub.

7. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	[x]
Dockerfile funcional	[x]
docker-compose.yml ou arquivos Kubernetes	[x]
Pipeline com etapas de build, teste e deploy	[x]
README.md com instruções e prints	[ ] marcar após inserir os prints
Documentação técnica com evidências (PDF ou PPT)	[ ] marcar após inserir os prints
Deploy realizado nos ambientes staging e produção	[ ] marcar após o pipeline rodar no GitHub